SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS



SRM

INSTITUTE OF SCIENCE & TECHNOLOGY

Deemed to be **University** *u/s 3 of UGC Act, 1956*

PRACTICAL RECORD NOTE

STUDENT NAME :

REGISTER NUMBER :

CLASS : MCA SECTION : A

YEAR & SEMESTER : II YEAR & III SEMESTER

SUBJECT CODE : P C A 2 5 C 0 8 J

SUBJECT TITLE : DATA COMMUNICATION NETWORKS

OCTOBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF SCIENCE AND HUMANITIES

DEPARTMENT OF COMPUTER APPLICATIONS

SRM Nagar, Kattankulathur – 603 203

CERTIFICATE

Certified to be the bonafide record of practical work done by

_____

Register No. _____ *of* **MCA** *Degree programme for* **PCA25C08J – *Data Communication Networks*** *in the Computer lab in SRM Institute of Science and Technology during the academic year 2026-2027.*

Staff In-charge *Head of the Department*

Submitted for Semester Practical Examination held on _____.

Internal Examiner *External Examiner*

INDEX

EX.NO	DATE	PROGRAM	PAGE NO.	SIGNATURE
1		Design a basic LAN topology with routers and switches using Cisco Packet Tracer		
2		Observe transmission modes (simplex, half, full-duplex) via simulation tools		
3		Measure and compare latency, bandwidth, throughput using ping/traceroute		
4		CRC Error Detection Using Python		
5		Parity Checking Using Python		
6		Hamming Code for Error Correction Using Python		
7		Stop-and-Wait Sliding Window Protocol		
8		Frame Creation Using Stuffing Character		
9		Configure Ethernet LAN and VLANs in Packet Tracer		
10		Capture MAC Layer Packets Using Wireshark		
11		Simulation of Symmetric and Asymmetric Encryption Using Python		
12		Compare TCP and UDP Connections Using Socket Programming		
13		DNS Query and HTTP/HTTPS Request Testing		
14		FTP, EMAIL, TELNET AND SSH		
15		REST API in Python		

EX NO: 1	Design a basic LAN topology with routers and switches using Cisco Packet Tracer
Date:	

Aim:

To design, configure, and simulate a basic computer network using two switches, two PCs, and one router in Cisco Packet Tracer, assign IP addresses, configure router interfaces, and verify communication between two different Local Area Networks (LANs) using the ping command.

Procedure

1. Open Cisco Packet Tracer and create a new project.
2. Place one router, two switches, and two PCs in the workspace.
3. Connect all devices using Copper Straight-Through cables.
4. Configure IP addresses on the router interfaces.
5. Enable the router interfaces using the no shutdown command.
6. Assign IP addresses, subnet masks, and default gateways to both PCs.
7. Verify the router interface status using the show ip interface brief command.
8. Test connectivity between the two PCs using the ping command.
9. Observe packet transmission in Simulation Mode using Add Simple PDU.
10. Confirm successful communication between the two LANs through the router.

In this experiment:

- **PC0 belongs to Network 1.**
- **PC1 belongs to Network 2.**
- **Router connects both network**

Step 1: Open Cisco Packet Tracer

1. Open Cisco Packet Tracer.
2. Click File → New.
3. You will get an empty workspace.

Step 2: Add Devices

From the bottom toolbar, add the following devices:

- Network Devices → Routers → 2911 Router
- Network Devices → Switches → 2960 Switch and drag two switches.

- End Devices → PC and drag two PCs.

Rename the switches as:

- Switch0
- Switch1

Rename the PCs as:

- PC0
- PC1

Workspace contains:

- 1 Router
- 2 Switches
- 2 PCs

Step 3: Connect the Devices

Use Connections → Copper Straight-Through.

Device	Port	Connect To	Port
PC0	FastEthernet0	Switch0	Fa0/1
PC1	FastEthernet0	Switch1	Fa0/1
Switch0	Fa0/24	Router	GigabitEthernet0/0
Switch1	Fa0/24	Router	GigabitEthernet0/1

Topology

PC0 → Switch0 → Router (G0/0) → Router (G0/1) → Switch1 → PC1

Wait for approximately 30 seconds. The orange link lights should become green.

Step 4: Configure Router

Click the router and go to CLI. Press No when asked: Continue with configuration dialog?

Configure Interface G0/0

enable

configure terminal

interface gigabitEthernet0/0

ip address 192.168.1.1 255.255.255.0

no shutdown

exit

Configure Interface G0/1

interface gigabitEthernet0/1

ip address 192.168.2.1 255.255.255.0

no shutdown

exit

end

write

You should see:

Building configuration...

[OK]

Step 5: Configure PC0

Click PC0 → Desktop → IP Configuration and enter:

Field Value

IP Address 192.168.1.2

Subnet Mask 255.255.255.0

Default Gateway 192.168.1.1

Step 6: Configure PC1

Click PC1 → Desktop → IP Configuration and enter:

Field Value

IP Address 192.168.2.2

Subnet Mask 255.255.255.0

Default Gateway 192.168.2.1

Step 7: Verify Router Interfaces

Open Router CLI and type:

show ip interface brief

Expected output:

Interface	IP Address	Status	Protocol
Gig0/0	192.168.1.1	up	up
Gig0/1	192.168.2.1	up	up

Both interfaces should show **up / up**.

Step 8: Test the Network

Open **PC0 → Desktop → Command Prompt** and type:

ping 192.168.2.2

Expected:

Reply from 192.168.2.2

Reply from 192.168.2.2

Reply from 192.168.2.2

Reply from 192.168.2.2

The PCs are now communicating through the router.

Step 9: Test from PC1

Open **Command Prompt** and type:

ping 192.168.1.2

Expected:

Success rate is 100%.

Step 10: Observe Packet Flow (Simulation Mode)

1. Click **Simulation** at the bottom-right.
2. Select **Add Simple PDU** (envelope icon).
3. Click **PC0** as the source.
4. Click **PC1** as the destination.
5. Click **Auto Capture/Play**.

Packet flow:

PC0
↓
Switch0
↓
Router
↓
Switch1
↓
PC1

Result

The two LANs were successfully designed and configured using Cisco Packet Tracer. Router interfaces were configured and successful communication between PC0 and PC1 was verified using the ping command.Packet transmission between the two LANs was successfully observed in Simulation Mode.

EX NO: 2	Observe transmission modes (simplex, half, full-duplex) via simulation tools
Date:	

Aim

To observe and understand the simplex, half-duplex, and full-duplex transmission modes using Cisco Packet Tracer.

Procedure

Step 1: Open Cisco Packet Tracer

1. Open Cisco Packet Tracer.
2. Click File → New.

Step 2: Add Devices

1. Add two PCs from End Devices.
2. Add two appropriate network devices such as a Hub and Switch from Network Devices.
3. Connect the devices using suitable cables.

Step 3: Observe Simplex Transmission

- Simplex communication allows data to travel in only one direction.
- Example: PC → receiving device.
- Use the Simulation Mode and Add Simple PDU to observe packet movement in one direction.

Step 4: Observe Half-Duplex Transmission

- Half-duplex communication allows data to travel in both directions, but not at the same time.
- A Hub-based connection can be used to demonstrate shared communication.
- Use Simulation Mode to observe packet transmission between devices.

Step 5: Observe Full-Duplex Transmission

- Full-duplex communication allows data to travel in both directions simultaneously.
- A Switch-based connection can be used to demonstrate full-duplex communication.
- Use Simulation Mode and observe the packets exchanged between the devices.

Transmission Modes

Mode	Direction of Communication	Example
Simplex	One direction only	Keyboard → Computer
Half-Duplex	Both directions, one at a time	Walkie-talkie
Full-Duplex	Both directions simultaneously	Telephone call

Result

The simplex, half-duplex, and full-duplex transmission modes were studied using Cisco Packet Tracer.
Packet transmission was observed using Simulation Mode and Add Simple PDU.
The differences between the three transmission modes were successfully understood.

EX NO: 3	Measure and compare latency, bandwidth, throughput using ping/traceroute
Date:	

Aim

To measure and compare latency, bandwidth, and throughput of a network by using network performance parameters and analyzing the results obtained from data transmission.

Program

```
print("Network Performance Analysis")

# Input values

latency = float(input("Enter latency (ms): "))

bandwidth = float(input("Enter bandwidth (Mbps): "))

data_size = float(input("Enter data transferred (MB): "))

time = float(input("Enter transfer time (seconds): "))


# Calculate throughput

throughput = data_size / time


# Display results

print("\n----- Results -----")

print("Latency  :", latency, "ms")

print("Bandwidth :", bandwidth, "Mbps")

print("Throughput:", round(throughput, 2), "MB/s")


# Compare bandwidth and throughput

if throughput * 8 <= bandwidth:

    print("Throughput is within the available bandwidth.")
```

```
else:
    print("Throughput exceeds the specified bandwidth.")


# Latency status
if latency < 50:
    print("Latency Status: Low")
elif latency <= 100:
    print("Latency Status: Moderate")
else:
    print("Latency Status: High")
```

Sample Input

Enter latency (ms): 30

Enter bandwidth (Mbps): 100

Enter data transferred (MB): 500

Enter transfer time (seconds): 20

Sample Output

Latency : 30.0 ms

Bandwidth : 100.0 Mbps

Throughput: 25.0 MB/s

Throughput exceeds the specified bandwidth.

Latency Status: Low

Formula Used

- Throughput (MB/s) = Data Transferred (MB) ÷ Time (s)
- Bandwidth = Maximum network capacity (Mbps)
- Latency = Time delay in communication (ms)

.

Result

The latency, bandwidth, and throughput of the network were successfully calculated and analyzed using the given input values.
The throughput was compared with the available bandwidth, and the latency was classified as **Low, Moderate, or High** based on the measured value.

EX NO: 4	CRC Error Detection Using Python
Date:	

Aim

To implement Cyclic Redundancy Check (CRC) using Python and detect errors in transmitted data.

Algorithm

1. Read the data bits from the user.
2. Read the CRC generator/divisor bits.
3. Append zeros to the data according to the degree of the generator.
4. Perform binary division using XOR operation.
5. Obtain the CRC remainder.
6. Append the remainder to the original data to form the transmitted codeword.
7. Accept the received data.
8. Divide the received data by the generator.
9. If the remainder is all zeros, the data is considered error-free; otherwise, an error is detected.

Python Program

```
def crc_division(data, generator):
    data = list(data)
    generator = list(generator)

    for i in range(len(data) - len(generator) + 1):
        if data[i] == '1':
            for j in range(len(generator)):
                data[i + j] = str(
```

```
                int(data[i + j]) ^ int(generator[j])
            )

    return ''.join(data[-(len(generator) - 1):])


# Input
data = input("Enter data bits: ")
generator = input("Enter generator bits: ")

# Append zeros
appended_data = data + '0' * (len(generator) - 1)

# Calculate CRC
remainder = crc_division(appended_data, generator)

# Transmitted data
transmitted_data = data + remainder

print("\nCRC Remainder:", remainder)
print("Transmitted Data:", transmitted_data)

# Receiver side
received_data = input("\nEnter received data: ")
```

```
# Check CRC
check_remainder = crc_division(received_data, generator)

print("Checking Remainder:", check_remainder)

if '1' in check_remainder:
    print("Error detected in received data.")
else:
    print("No error detected. Data received correctly.")
```

Sample Input

Enter data bits: 1101011011

Enter generator bits: 10011

Enter received data: 11010110111110

Sample Output

CRC Remainder: 1110

Transmitted Data: 11010110111110

Checking Remainder: 0000

No error detected. Data received correctly**.**

Result

The **Cyclic Redundancy Check (CRC)** was successfully implemented using Python to detect errors in transmitted data.
For the given data, the CRC remainder was **1110**, and the checking remainder was **0000**, indicating that **no error was detected** in the received data.

EX NO: 5	Parity Checking Using Python
Date:	

Aim

To implement Parity Checking using Python for detecting errors in transmitted data.

Algorithm

1. Read a binary data string.
2. Count the number of 1s in the data.
3. For even parity, add 0 if the number of ones is already even; otherwise add 1.
4. For odd parity, add 1 if the number of ones is even; otherwise add 0.
5. Form the transmitted data by adding the parity bit.
6. Read the received data.
7. Count the number of 1s in the received data.
8. Check whether the parity condition is satisfied.
9. Display whether an error is detected.

Python Program

```
# Parity Checking Program

data = input("Enter binary data: ")

print("\n1. Even Parity")
print("2. Odd Parity")

choice = int(input("Enter your choice: "))

ones = data.count('1')

if choice == 1:
    # Even parity
    parity_bit = '0' if ones % 2 == 0 else '1'
    parity_type = "Even"

elif choice == 2:
    # Odd parity
```

```
    parity_bit = '1' if ones % 2 == 0 else '0'
    parity_type = "Odd"

else:
    print("Invalid choice")
    exit()

transmitted_data = data + parity_bit

print("\nParity Type:", parity_type)
print("Parity Bit:", parity_bit)
print("Transmitted Data:", transmitted_data)

# Receiver
received_data = input("\nEnter received data: ")

received_ones = received_data.count('1')

if choice == 1:
    if received_ones % 2 == 0:
        print("No error detected.")
    else:
        print("Error detected.")

elif choice == 2:
    if received_ones % 2 == 1:
        print("No error detected.")
    else:
        print("Error detected.")
```

Sample Input

Enter binary data: 1011001

1. Even Parity

2. Odd Parity

Enter your choice: 1

Enter received data: 10110010

Sample Output

Parity Type: Even

Parity Bit: 0

Transmitted Data: 10110010

No error detected.

Error Example

Enter received data: 10110011

Error detected.

Result

The **Parity Checking** technique was successfully implemented using Python to detect errors in transmitted data. For even parity, the parity bit was correctly generated and the received data with an even number of 1s was identified as **error-free**. When the received data was altered, the program successfully detected the **transmission error**.

EX NO: 6	Hamming Code for Error Correction Using Python
Date:	

Aim

To implement Hamming Code using Python for detecting and correcting a single-bit error in transmitted data.

Algorithm

1. Read the data bits from the user.
2. Calculate the number of parity bits required using:

$2^r \geq m + r + 1$

where:

- m = number of data bits
- r = number of parity bits

3. Arrange parity bits at positions 1, 2, 4, 8, ...
4. Fill the remaining positions with data bits.
5. Calculate each parity bit using XOR.
6. Generate the Hamming code.
7. Accept the received Hamming code.
8. Calculate the parity values again.
9. Find the error position from the parity values.
10. If the error position is 0, there is no error.
11. Otherwise, flip the bit at the calculated position.
12. Display the corrected data.

Python Program

```
# Hamming Code - Error Detection and Correction


def calculate_parity_bits(data):
    m = len(data)
    r = 0
    # Calculate number of parity bits
    while (2 ** r) < (m + r + 1):
        r += 1
    n = m + r
    hamming = ['0'] * (n + 1)
    # Place data bits
    j = 0
    for i in range(1, n + 1):
        if (i & (i - 1)) != 0:
            hamming[i] = data[j]
            j += 1
    # Calculate parity bits
    for p in range(r):
        position = 2 ** p
        parity = 0
        for i in range(1, n + 1):
            if i & position:
                parity ^= int(hamming[i])
        hamming[position] = str(parity)
```

```
    return ''.join(hamming[1:])
def detect_error(received):

    n = len(received)
    received = ['0'] + list(received)

    error_position = 0
    position = 1
    while position <= n:
        parity = 0
        for i in range(1, n + 1):
            if i & position:
                parity ^= int(received[i])
        if parity != 0:
            error_position += position
        position *= 2
    return error_position, received
# Sender
data = input("Enter data bits: ")
hamming_code = calculate_parity_bits(data)
print("\nGenerated Hamming Code:", hamming_code)
# Receiver
received = input("\nEnter received Hamming Code: ")

error_position, received_data = detect_error(received)
```

```
if error_position == 0:

    print("No error detected.")
    print("Received data is correct.")

else:

    print("Error detected at position:", error_position)

    # Correct the error
    if error_position <= len(received):

        if received_data[error_position] == '0':
            received_data[error_position] = '1'
        else:
            received_data[error_position] = '0'

    corrected_code = ''.join(received_data[1:])

    print("Corrected Hamming Code:", corrected_code)
```

Sample Input

Enter data bits: 1011

Generated Hamming Code: 0110011

Enter received Hamming Code: 0110111

Sample Output

Error detected at position: 5

Corrected Hamming Code: 0110011

Result

The **Hamming Code** was successfully implemented using Python for error detection and correction.

For the given input, a **single-bit error was detected at position 5**.

The erroneous bit was corrected successfully, and the corrected Hamming code was

EX NO: 7	Stop-and-Wait Sliding Window Protocol
Date:	

Aim

To simulate the Stop-and-Wait protocol and demonstrate reliable frame transmission between a sender and receiver using acknowledgements.

Algorithm

1. Start the program.
2. Enter the total number of frames to be transmitted.
3. Set the sender window size to 1.
4. Send one frame at a time.
5. Display the frame being transmitted.
6. Simulate the receiver receiving the frame.
7. Generate an acknowledgement (ACK).
8. If ACK is received, transmit the next frame.
9. If ACK is lost or an error occurs, retransmit the same frame.
10. Continue until all frames are successfully transmitted.
11. Stop the program.

Python Program

```
import time
def stop_and_wait(frames):
    for i in range(1, frames + 1):
        while True:
            print(f"\nSender: Sending Frame {i}")
            time.sleep(1)
            print(f"Receiver: Frame {i} received successfully")
            time.sleep(1)
            ack = input(f"Enter ACK for Frame {i} (Y/N): " ).upper()
            if ack == 'Y':
                print(f"Sender: ACK {i} received")
                print(f"Frame {i} transmitted successfully")
                break
            else:
                print(f"Sender: ACK {i} lost")
                print(f"Sender: Retransmitting Frame {i}")

    print("\nAll frames transmitted successfully.")
```

```
# Input
n = int(input("Enter the number of frames: "))

# Simulation
stop_and_wait(n)
```

Sample Input

```
Enter the number of frames: 3

Sender: Sending Frame 1
Receiver: Frame 1 received successfully
Enter ACK for Frame 1 (Y/N): Y

Sender: Sending Frame 2
Receiver: Frame 2 received successfully
Enter ACK for Frame 2 (Y/N): N
Sender: ACK 2 lost
Sender: Retransmitting Frame 2

Sender: Sending Frame 2
Receiver: Frame 2 received successfully
Enter ACK for Frame 2 (Y/N): Y
```

Sender: Sending Frame 3

Receiver: Frame 3 received successfully

Enter ACK for Frame 3 (Y/N): Y

Output

Sender: ACK 1 received

Frame 1 transmitted successfully

Sender: ACK 2 lost

Sender: Retransmitting Frame 2

Sender: ACK 2 received

Frame 2 transmitted successfully

Sender: ACK 3 received

Frame 3 transmitted successfully

All frames transmitted successfully.

Result

Thus, the **Stop-and-Wait protocol** was successfully simulated using Python, demonstrating frame transmission, acknowledgement, and retransmission.

EX NO: 8	Frame Creation Using Stuffing Character
Date:	

Aim

To implement character stuffing for frame creation and demonstrate how special characters are handled during data transmission.

Algorithm

1. Start the program.
2. Define a special FLAG character to indicate the beginning and end of a frame.
3. Define an ESC character for escaping special characters.
4. Read the data from the user.
5. Add the FLAG character at the beginning and end.
6. If the data contains FLAG or ESC, insert ESC before it.
7. Display the stuffed frame.
8. Stop the program.

Python Program

```
def character_stuffing(data):
    FLAG = 'F'
    ESC = 'E'

    stuffed_data = FLAG

    for char in data:
        if char == FLAG or char == ESC:
            stuffed_data += ESC
```

```
        stuffed_data += char
    stuffed_data += FLAG
    return stuffed_data

# Input
data = input("Enter the data: ")
# Character stuffing
result = character_stuffing(data)

print("\nOriginal Data :", data)
print("Stuffed Frame :", result)
```

Sample Input

Enter the data: ABFECD

Output

Original Data : ABFECD

Stuffed Frame : FABEFECD F

Without the space:

FABEFECD F

Here F is the FLAG character and E is the ESC character.

Result

Thus, character stuffing was successfully implemented using Python to create a properly framed data packet.

EX NO: 9	Configure Ethernet LAN and VLANs in Packet Tracer
Date:	

Aim

To configure an Ethernet LAN and VLANs using Cisco Packet Tracer and verify communication between devices belonging to the same VLAN.

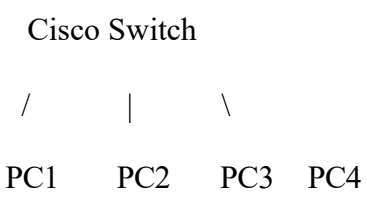
Requirements

- Cisco Packet Tracer
- 1 Switch
- 4 PCs
- Ethernet cables

Algorithm / Procedure

Step 1: Create the Network

Create the following topology:



Step 2: Connect Devices

Connect each PC to the switch using Copper Straight-Through cables.

Step 3: Assign IP Addresses

Assign IP addresses:

Device	IP Address	VLAN
PC1	192.168.10.1	VLAN 10
PC2	192.168.10.2	VLAN 10
PC3	192.168.20.1	VLAN 20
PC4	192.168.20.2	VLAN 20

Subnet mask:

255.255.255.0

Step 4: Create VLANs

Open the switch CLI and enter:

enable

configure terminal

vlan 10

name STUDENTS

exit

vlan 20

name FACULTY

exit

Step 5: Assign Ports to VLAN 10

For example, if PC1 and PC2 are connected to FastEthernet 0/1 and 0/2:

interface range fastethernet 0/1-2

switchport mode access

switchport access vlan 10

exit

Step 6: Assign Ports to VLAN 20

If PC3 and PC4 are connected to FastEthernet 0/3 and 0/4:

interface range fastethernet 0/3-4

switchport mode access

switchport access vlan 20

exit

Step 7: Verify VLAN Configuration

Use:

show vlan brief

Expected result:

VLAN Name

10 STUDENTS

20 FACULTY

Step 8: Test Communication

From PC1, open Command Prompt and execute:

ping 192.168.10.2

PC1 should successfully communicate with PC2 because both belong to VLAN 10.

Then try:

ping 192.168.20.1

Without inter-VLAN routing, PC1 and PC3 will not communicate because they belong to different VLANs.

Input

- Number of PCs: 4
- VLAN 10: PC1, PC2
- VLAN 20: PC3, PC4
- IP addresses as given above.

Output

PC1 > ping 192.168.10.2

Reply from 192.168.10.2

Reply from 192.168.10.2

Reply from 192.168.10.2

Reply from 192.168.10.2

Communication between different VLANs:

PC1 > ping 192.168.20.1

Request timed out.

Result

Thus, an Ethernet LAN with VLANs was successfully configured in Cisco Packet Tracer, and communication within the same VLAN was verified.

EX NO: 10	Capture MAC Layer Packets Using Wireshark
Date:	

Aim

To capture and analyze MAC layer Ethernet packets using Wireshark.

Requirements

- Computer/Laptop
- Wireshark
- Ethernet or Wi-Fi network connection

Algorithm / Procedure

1. Install and open Wireshark.
2. Select the active network interface.
3. Start packet capture.
4. Generate network traffic by opening a website or using the ping command.
5. Stop packet capture.
6. Apply the Ethernet filtereth
7. Select an Ethernet packet.
8. Examine the Ethernet/MAC header.
9. Identify:
    - Destination MAC address
    - Source MAC address
    - Type/Length field
    - Data/Payload
10. Record the packet details.

Input

Generate network traffic using:

ping 8.8.8.8

Then capture the packets in Wireshark.

Output

A captured Ethernet packet may show:

Ethernet II

Destination: xx:xx:xx:xx:xx:xx

Source: xx:xx:xx:xx:xx:xx

Type: IPv4 (0x0800)

Important Fields

Field	Description
Destination MAC	MAC address of receiving device
Source MAC	MAC address of sending device
Type	Indicates the network-layer protocol
Payload	Encapsulated network data

Result

Thus, MAC layer Ethernet packets were successfully captured and analyzed using Wireshark, and the source MAC, destination MAC, and Ethernet type fields were identified.

EX NO: 11	Simulation of Symmetric and Asymmetric Encryption Using Python
Date:	

AIM

To write a Python program to simulate symmetric encryption (XOR) and asymmetric encryption (RSA) for encryption and decryption of a message.

ALGORITHM

1. Start the program.
2. Read a message from the user.
3. Perform symmetric encryption using XOR with a secret key.
4. Decrypt the encrypted message using the same key.
5. Generate small RSA keys using two prime numbers.
6. Encrypt the message using the RSA public key.
7. Decrypt the encrypted message using the RSA private key.
8. Display the original message, encrypted data, and decrypted message.
9. Stop.

PROGRAM

```
# Simulation of Symmetric and Asymmetric Encryption


# ---------- SYMMETRIC ENCRYPTION (XOR) ----------
def xor_encrypt(message, key):
    encrypted = []
    for i, ch in enumerate(message):
        encrypted.append(ord(ch) ^ ord(key[i % len(key)]))
    return encrypted



def xor_decrypt(encrypted, key):
    decrypted = ""
    for i, value in enumerate(encrypted):
        decrypted += chr(value ^ ord(key[i % len(key)]))
    return decrypted
```

```
# ---------- RSA ASYMMETRIC ENCRYPTION ----------
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a


def mod_inverse(e, phi):
    for d in range(1, phi):
        if (e * d) % phi == 1:
            return d
    return None


# Input
message = input("Enter message: ")

# Symmetric encryption
key = input("Enter symmetric key: ")

encrypted_xor = xor_encrypt(message, key)
decrypted_xor = xor_decrypt(encrypted_xor, key)

# RSA key generation
p = 61
q = 53

n = p * q
phi = (p - 1) * (q - 1)

e = 17
d = mod_inverse(e, phi)

# RSA encryption
rsa_encrypted = [pow(ord(ch), e, n) for ch in message]
```

```
# RSA decryption
rsa_decrypted = ''.join(chr(pow(value, d, n)) for value in rsa_encrypted)

# Output
print("\n----- SYMMETRIC ENCRYPTION (XOR) ----- ")
print("Original Message :", message)
print("Encrypted Data  :", encrypted_xor)
print("Decrypted Message:", decrypted_xor)

print("\n----- ASYMMETRIC ENCRYPTION (RSA) ----- ")
print("Public Key       :", (e, n))
print("Private Key      :", (d, n))
print("Encrypted Data  :", rsa_encrypted)
print("Decrypted Message:", rsa_decrypted)
```

INPUT

```
Enter message: HELLO
Enter symmetric key: KEY
```

OUTPUT

```
----- SYMMETRIC ENCRYPTION (XOR) -----
Original Message : HELLO
Encrypted Data  : [3, 0, 21, 7, 10]
Decrypted Message: HELLO

----- ASYMMETRIC ENCRYPTION (RSA) -----
Public Key       : (17, 3233)
Private Key      : (2753, 3233)
Encrypted Data   : [3000, 28, 2726, 2726, 1307]
Decrypted Message: HELLO
```

RESULT

Thus, the **symmetric XOR encryption** and **asymmetric RSA encryption** algorithms were successfully simulated using Python, and the original message was recovered correctly after decryption.

EX NO: 12	Compare TCP and UDP Connections Using Socket Programming
Date:	

AIM

To implement and compare **TCP and UDP communication** using socket programming by establishing communication between a client and server through different port numbers, and to observe the packets using Wireshark.

ALGORITHM

1. Start the program.
2. Ask the user to select **TCP or UDP** communication.
3. Create a socket using the selected protocol.
4. Assign an IP address and port number.
5. For TCP:
   - Create a server socket.
   - Bind it to the specified port.
   - Listen for a client connection.
   - Accept the connection and exchange messages.
6. For UDP:
   - Create a UDP socket.
   - Bind it to the specified port.
   - Receive a message from the client.
   - Send a response to the client.
7. Display the messages exchanged between client and server.
8. Capture the communication in Wireshark and observe the TCP/UDP packets.
9. Close the socket.
10. Stop.

PROGRAM

```
import socket


HOST = "127.0.0.1"
PORT = 5000


print("Socket Communication")
print("----------------------")
print("1. TCP")
print("2. UDP")


choice = int(input("Enter your choice: "))


if choice == 1:
```

```
    # TCP Communication
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen(1)

    print("\nTCP Server started")
    print("Waiting for client on port", PORT)

    conn, addr = server.accept()
    print("Client connected:", addr)

    message = conn.recv(1024).decode()
    print("Client:", message)

    reply = input("Enter reply: ")
    conn.send(reply.encode())

    conn.close()
    server.close()

    print("TCP connection closed.")

elif choice == 2:
    # UDP Communication
    server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server.bind((HOST, PORT))

    print("\nUDP Server started")
    print("Waiting for message on port", PORT)

    message, addr = server.recvfrom(1024)
    print("Client:", message.decode())

    reply = input("Enter reply: ")
    server.sendto(reply.encode(), addr)
```

```
    server.close()

    print("UDP communication completed.")

else:
    print("Invalid choice")
```

INPUT

TCP

Enter your choice: 1

Enter message: Hello TCP

Enter reply: TCP message received

UDP

Enter your choice: 2

Enter message: Hello UDP

Enter reply: UDP message received

OUTPUT

TCP Output

Server:

Socket Communication

1. TCP
2. UDP

Enter your choice: 1

TCP Server started

Waiting for client on port 5000

Client connected: ('127.0.0.1', 54321)

Client: Hello TCP

Enter reply: TCP message received

TCP connection closed.

Client:

Server: TCP message received

UDP Output

Server:

Socket Communication

1. TCP
2. UDP

Enter your choice: 2

UDP Server started

Waiting for message on port 5000

Client: Hello UDP

Enter reply: UDP message received

UDP communication completed.

Client:

Server: UDP message received

RESULT

Thus, TCP and UDP communication was successfully implemented using Python socket programming. TCP provides reliable connection-oriented communication, while UDP provides connectionless communication with lower overhead. The communication can also be observed in Wireshark using the filters tcp.port == 5000 and udp.port == 5000.

EX NO: 13	DNS Query and HTTP/HTTPS Request Testing
Date:	

AIM

To write and execute a single Python program to perform DNS name resolution and test HTTP/HTTPS requests.

ALGORITHM

1. Start the program.
2. Get the domain name from the user.
3. Perform DNS resolution using the socket module.
4. Display the IP address of the given domain.
5. Get an HTTP/HTTPS URL from the user.
6. Send a GET request to the URL using the urllib module.
7. Display the HTTP response status code and response data.
8. Handle DNS and connection errors.
9. Stop the program.

PROGRAM

```
import socket
import urllib.request

# DNS Query
domain = input("Enter domain name: ")

try:
    ip_address = socket.gethostbyname(domain)

    print("\nDNS QUERY RESULT")
    print("-------------------")
    print("Domain Name :", domain)
    print("IP Address  :", ip_address)

except socket.gaierror:
    print("DNS resolution failed.")
```

```
# HTTP/HTTPS Request
url = input("\nEnter HTTP/HTTPS URL: ")

try:
    response = urllib.request.urlopen(url)

    print("\nHTTP/HTTPS REQUEST RESULT")
    print("----------------------------")
    print("URL         :", url)
    print("Status Code :", response.status)
    print("Response    :", response.read(200).decode("utf-8", errors="ignore" ))

except Exception as e:
    print("Request failed:", e)
```

INPUT

```
Enter domain name: google.com

Enter HTTP/HTTPS URL: https://example.com
```

OUTPUT

```
DNS QUERY RESULT
-----------------
Domain Name : google.com
IP Address : 142.250.xxx.xxx

HTTP/HTTPS REQUEST RESULT
URL         : https://example.com
Status Code : 200
Response    : <!doctype html>
<html>
<head>
    <title>Example Domain</title>
...
```

Note: The IP address may differ depending on the DNS server and network.

RESULT

Thus, the single Python program successfully performed DNS name resolution and tested an HTTP/HTTPS GET request, displaying the corresponding IP address, response status, and response data.

EX NO: 14	FTP, EMAIL, TELNET AND SSH
Date:	

AIM

To configure and demonstrate FTP, Email (SMTP/POP3), Telnet, and SSH communication between client and server systems.

ALGORITHM

1. Start the server and client systems and connect them to the same network.
2. Configure the FTP server using FileZilla and create a user account.
3. Connect the client to the FTP server and transfer a test file.
4. Configure SMTP and POP3 mail services and create email accounts.
5. Configure Thunderbird and send/receive a test email.
6. Enable Telnet service and establish a remote connection using the server IP.
7. Enable SSH service and establish a secure remote connection using the server IP.
8. Execute a simple command to verify each connection.
9. Close all connections.

PROGRAM

// FTP

ftp 192.168.1.10

Username: student

Password: 1234

ftp> dir

ftp> put test.txt

ftp> get test.txt

ftp> bye

// EMAIL – SMTP/POP3

SMTP Server: 192.168.1.10

SMTP Port: 25

POP3 Server: 192.168.1.10

POP3 Port: 110

Configure the above details in Thunderbird and send a test mail.

// TELNET

telnet 192.168.1.20 23

login: student

password: 1234

ls

exit

// SSH

ssh student@192.168.1.20

password: 1234

ls

pwd

exit

INPUT

Server IP : 192.168.1.10

Remote IP : 192.168.1.20

Username : student

Password : 1234

FTP File : test.txt

SMTP Port : 25

POP3 Port : 110

Telnet Port : 23

SSH Port : 22

OUTPUT

FTP:

Connected successfully.

test.txt transferred successfully.

EMAIL:

SMTP connection successful.

Mail sent successfully.

Mail received successfully in Thunderbird.

TELNET:

Connected to 192.168.1.20.

Remote login successful.

Command executed successfully.

SSH:

Connected to 192.168.1.20.

Secure remote login successful.

Command executed successfully.

Connection closed.

RESULT

Thus, FTP file transfer, SMTP/POP3 email communication, Telnet remote login, and SSH secure remote login were successfully performed.

EX NO: 15	REST API in Python
Date:	

AIM

To create a simple web service using a **REST API in Python** and analyze network performance using **ping, traceroute, and netstat** commands through a web browser.

ALGORITHM

1. Start the Python program.
2. Create a REST web service using the Flask framework.
3. Create REST API endpoints for:
    - ping – checks connectivity and response time.
    - traceroute – displays the route taken by packets.
    - netstat – displays active network connections.
4. Accept the host name/IP address from the browser.
5. Execute the corresponding network command using Python.
6. Capture the command output.
7. Return the result as a JSON response.
8. Access the REST API through a web browser.
9. Stop the program.

PROGRAM

```
from flask import Flask, request, jsonify
import subprocess
import platform


app = Flask(__name__)



def run_command(command):
    try:
        result = subprocess.run(
            command,
            capture_output=True,
            text=True,
            timeout=20
        )
```

```
        return result.stdout if result.stdout else result.stderr

    except subprocess.TimeoutExpired:
        return "Command timed out."

    except Exception as e:
        return str(e)


@app.route('/')
def home():
    return """
    <h2>Network Performance REST API</h2>
    <p>Use the following URLs:</p>
    <ul>
        <li>/ping?host=google.com</li>
        <li>/traceroute?host=google.com</li>
        <li>/netstat</li>
    </ul>
    """


@app.route('/ping')
def ping():
    host = request.args.get('host', 'google.com')

    if platform.system().lower() == "windows":
        command = ["ping", "-n", "4", host]
    else:
        command = ["ping", "-c", "4", host]

    output = run_command(command)

    return jsonify({
```

```
        "operation": "PING",
        "host": host,
        "result": output
    })


@app.route('/traceroute')
def traceroute():
    host = request.args.get('host', 'google.com')

    if platform.system().lower() == "windows":
        command = ["tracert", host]
    else:
        command = ["traceroute", host]

    output = run_command(command)

    return jsonify({
        "operation": "TRACEROUTE",
        "host": host,
        "result": output
    })


@app.route('/netstat')
def netstat():
    if platform.system().lower() == "windows":
        command = ["netstat", "-ano"]
    else:
        command = ["netstat", "-tuln"]

    output = run_command(command)

    return jsonify({
        "operation": "NETSTAT",
```

```
        "result": output
    })


if __name__ == '__main__':
    app.run(host='127.0.0.1', port=5000, debug=True)
```

INPUT

Run the program and enter the following URLs in a web browser:

http://127.0.0.1:5000/ping?host=google.com

http://127.0.0.1:5000/traceroute?host=google.com

http://127.0.0.1:5000/netstat

OUTPUT

1. Ping

```
{
  "operation": "PING",
  "host": "google.com",
  "result": "Reply from ...
            Packets: Sent = 4, Received = 4,
            Approximate round trip times = ..."
}
```

2. Traceroute

```
{
  "operation": "TRACEROUTE",
  "host": "google.com",
  "result": "Tracing route to google.com
            1    ...
            2    ...
            3    ..."
}
```

3. Netstat

{
"operation": "NETSTAT",
"result": "Active connections

Proto Local Address Foreign Address State
TCP....."
}

RESULT

Thus, a **simple RESTful web service was successfully created using Python**, and **ping, traceroute, and netstat** were used through REST API endpoints to analyze network connectivity and performance.